SYSTEM ARCHITECT AUTOMOTIVE

GIÁO TRÌNH CHUYÊN SÂU
DIAGNOSTIC TX DEMO

Luồng truyền dữ liệu UDS dài qua CanTp

Biên soạn: **Phạm Văn Vũ**

HALA Academy

GIÁO TRÌNH CHUYÊN SÂU AUTOSAR DIAGNOSTIC TX (CANTP)

Băm dữ liệu lớn thành mảng nhỏ qua CanTp và PduR

> Tài liệu này giải phẫu toàn bộ kiến trúc truyền tin chẩn đoán ở Node Transmitter (TX), mô phỏng một Tester gửi tải trọng (payload) UDS 20 bytes thông qua chuẩn mạng CAN vốn dĩ giới hạn ở 8 bytes mỗi khung bằng sự hỗ trợ của ISO-TP (CanTp).

Bản chất sự việc: Chuẩn giao tiếp chẩn đoán như UDS (ISO 14229) hay yêu cầu truyền và nhận những khối lượng dữ liệu khổng lồ (ví dụ Flash Bootloader, Read DTC). Trong khi đó Data Field của CAN dài kịch kim chỉ 8 bytes. AUTOSAR `CanTp` (CAN Transport Layer) khắc phục giới hạn vật lý này bằng việc băm nhỏ Data ra nhiều **Consecutive Frames (CF)** theo sau 1 nhịp thông báo khởi đầu **First Frame (FF)** và sự xác thực bằng **Flow Control (FC)**.

Mục lục

1. Mục tiêu học tập và bức tranh toàn cục
2. Cấu trúc ISO-TP và CAN Tp TX
3. Quy chiếu ID xuống nền tảng mã nguồn
4. Cơ chế GDB Python - Tool test tự động
5. Chi tiết quá trình Runtime của TX Node
6. Biểu đồ kỹ thuật: State Machine & Sequence

---

1. Mục tiêu học tập và bức tranh toàn cục

Phần này tập trung chuyên biệt vào quá trình gửi Dữ liệu chẩn đoán từ tầng Ứng Dụng đi xuống mạng CAN Bus. Chúng ta rút ra các đặc tính:

- Tại sao lại xuất hiện khái niệm First Frame (FF) và Consecutive Frame (CF)?
- State Machine trong `CanTp_MainFunction()` điều hành các công việc này độc lập với Application ra sao?
- PduR chỉ đơn thuần route như bản thân nó ở COM Pdu, hay nó có nhiệm vụ báo cáo kết quả?
- GDB Script can thiệp thẳng vào hệ thống giả lập Renode bằng cách nào để biến một ứng dụng tĩnh thành Testing Script.

> **Chuỗi đi đúng của dữ liệu trong luồng TX:**
> `GDB Trigger -> diag_tx_demo.c -> CanTp_Transmit() -> CanTp_MainFunction (WAIT_FC) -> CanIf_Transmit (FF) -> Mạng CAN -> Can_MainFunction_Read (nhận FC) -> CanTp_MainFunction (WAIT_CF_TX) -> CanIf_Transmit (CF) -> Hoàn tất.`

---

2. Cấu trúc ISO-TP và CAN Tp TX

2.1 ISO-TP PDU Types (Các loại khung vận chuyển)

Để băm một mảng 20 bytes ra thành các frame CAN 8 bytes, AUTOSAR thiết đặt các chỉ mục Header 1 byte hoặc 2 bytes phía trước Data thật.

Kiểu Khung	Nibble Ký Hiệu Đầu (N_PCI)	Vai Trò
Single Frame (SF)	`0x0`	Dùng khi dữ liệu nhỏ bé (<= 7 bytes trên CAN chuẩn). Khung tự thân đầy đủ, không cần băm mảnh.
First Frame (FF)	`0x1`	Khung tiên phong mang tổng chiều dài dữ liệu (VD: 20 bytes) để Node RX kịp cấp phát/chuẩn bị Buffer đủ bự.
Consecutive Frame (CF)	`0x2`	Mang theo dữ liệu băm nhỏ (thường 7 bytes mỗi CF). Kèm luôn cả số thứ tự Serial Number (SN) ở Nibble thứ 2 để chống sót.
Flow Control (FC)	`0x3`	Lệnh "Điều khiển luồng" gửi về từ Node RX. BlockSize (BS) và STmin thông báo TX được gửi dồn bao nhiêu gói CF và độ trễ chờ tối thiểu (STmin).

2.2 Cơ Chế State Machine Trong CanTp TX

Chính vì phải chờ lệnh FC điều phối từ Node nhận (tránh làm tràn bộ đệm phần cứng của RX Node khi nhồi CF quá nhanh), CanTp thiết lập cỗ máy trạng thái State Machine riêng đè lên luồng truyền đi thay vì ném một phát thẳng vào CanIf.

Trạng Thái Trong Code	Ý Nghĩa Diễn Giải
`CANTP_TX_IDLE`	Hệ thống nằm chờ API gọi đến để cấp phát Buffer.
`CANTP_TX_WAIT_FC`	Đã tống khứ FF đi. Thắt đai an toàn, thiết lập Timer đếm ngược (N_Bs) chờ phản hồi FC từ Node RX. Nếu mốc Timer này nổ ra Timeout -> Báo lỗi huỷ truyền.
`CANTP_TX_WAIT_CF`	Cầm trong tay kim bài FC. Bắt đầu đẩy đống CF ra Buffer truyền. Máy đếm Consecutive Frame bắt đầu rải đạn vòng lặp. Nếu Block Size chạm max (BS>0), sẽ rớt về lại trạng thái `WAIT_FC`.

---

3. Quy chiếu ID xuống nền tảng mã nguồn

Bản demo chạy `diag_tx_demo.c` chứa một luồng dữ liệu 20 byte tĩnh được ép liên thông vào một PduId TX duy nhất.

Tên Module / Thành Phần	Tên Macro/Hằng Số PDU	Kích thước và ID Giá Trị
CanTp Cfg	`CanTpConf_CanTpTxNSdu_DiagTx`	Tx PduId = `0` ; RxFc PduId = `1` ; N_As = `1000` ms; N_Bs = `1000` ms; N_Cs = `0` ms
CanIf Cfg	`CanIfConf_Pdu_DiagTx` / `CanIfConf_Pdu_DiagRx_FC`	Tx CAN ID = `0x7DF` (Functional) / FC CAN ID = `0x7E8` (Physical)

> **Nguyên lý ID đảo nghịch chẩn đoán:** TX truyền dữ liệu ra với ID = `0x7DF`. Node RX nhận được frame có ID đó, và khi báo Flow Control, Node RX truyền lại ngược luồng bằng ID `0x7E8`. Nên trên mặt trận Node TX, nó phải cấu hình 1 **Bộ lọc CanIf RX ID = 0x7E8** để hứng được Flow Control!

---

4. Cơ chế GDB Python - Tool test tự động

Để hoàn toàn tự động hóa việc Tester nhấn nút phát dữ liệu chẩn đoán, một Python script được sinh ra. Bằng việc chọc sâu vào lõi của phần mềm mô phỏng Renode Emulator thông qua cánh cổng GDB Server (Cổng 3333).

```
# Đoạn mã Python đính kèm GDB tương tác với diag_tx_demo.elf
gdb_commands = """
target remote :3333
# Thiết lập biến giả lập người ấn nút truyền dữ liệu
set variable trigger_diag = 1
# Tiếp tục chạy thực thi
c
"""
```

Tại vòng lặp `main()` của `diag_tx_demo.c`, biến tĩnh `trigger_diag` được vòng lặp Polling liên tục kiểm tra. Ngay khi GDB nện giá trị 1 lên vùng RAM đó, cờ hiệu bốc cháy, lệnh `CanTp_Transmit(0, &pduInfo)` được rải qua, và GDB lập tức set cờ về 0 để đóng van chống spam.

---

5. Chi tiết quá trình Runtime của TX Node

5.1 Ứng dụng phát lệnh `CanTp_Transmit`

```
// Mảng dữ liệu mồi
static uint8 udsData[20] = {
    0x22, 0xF1, 0x90, 0x11, 0x22, 0x33, 0x44, 0x55,
    // ... payload kéo dài 20 bytes
};

// Gọi phát nếu Trigger
if (trigger_diag) {
    trigger_diag = 0; // Tắt chốt kích hoạt
    PduInfoType pduInfo;
    pduInfo.SduDataPtr = udsData;
    pduInfo.SduLength = sizeof(udsData);

    CanTp_Transmit(CanTpConf_CanTpTxNSdu_DiagTx, &pduInfo);
}
```

5.2 Logic Cắt Ghép Phân Luồng của First Frame

Tại hàm `CanTp_Transmit()` trong `CanTp.c`, mã nguồn tính toán SduLength. Vì nó 20 bytes lớn hơn dư dả 7 bytes. Nó chuyển trạng thái của Node TX sang mô hình **Băm Mảnh (Segmentation)**.

```
// CanTp.c – Hàm logic đóng ngắt sinh FF
if (len > 7) {
    ch->txBytesRemaining = len;
    ch->txSduPtr = PduInfoPtr->SduDataPtr;

    // Lắp ghép PCI N_PCItype = 1 (First Frame)
    uint8 ff_pci[2];
    ff_pci[0] = 0x10 | ((len >> 8) & 0x0F);
    ff_pci[1] = len & 0xFF;

    // Nhét 6 byte data đầu tiên vào đuôi FF
    uint8 frameData[8];
    frameData[0] = ff_pci[0];
    frameData[1] = ff_pci[1];
    for (int i=0; i<6; ++i) frameData[2+i] = ch->txSduPtr[i];

    // Gửi đi bằng CanIf! Cầm vô-lăng CAN ID 0x7DF
    CanIf_Transmit(CanTp_Cfg_Tx[id].CanIfTxPduId, &ffPdu);

    // Khóa trạng thái, đón lệnh Flow Control hồi âm!
    ch->txState = CANTP_TX_WAIT_FC;
    ch->txSn = 1; // SN kế tiếp CF là 1
}
```

5.3 Tiếp Chiêu Trả Lời Của Receiver Bằng Ngắt Polling (FC)

Bên trạm thu ngửi được mùi First Frame 20 Bytes, nó sinh ra Buffer cấp phát và vác bộ đàm gọi tín hiệu Flow Control mang theo Clear-to-Send (CTS). Bộ đàm dùng bước sóng (CAN ID) `0x7E8` dội thẳng vào CAN Receiver chân thu của trạm phát.

Polling vòng lặp của ta có tích hợp `Can_MainFunction_Read()` đã lắng nghe và chộp được. Nó tra Filter của CanIf, thấy ID `0x7E8`, gọi hàm: `CanTp_RxIndication` của chẩn đoán.

```
// Chẻ mã PCI Type xem có phải FC không?
if (pciType == 0x3) {
    // Trạng thái (CTS, WAIT, OVERFLOW), BlockSize, STMin
    uint8 fs = PduInfoPtr->SduDataPtr[0] & 0x0F;
    uint8 bs = PduInfoPtr->SduDataPtr[1];
    uint8 stmin = PduInfoPtr->SduDataPtr[2];

    if (fs == 0 && ch->txState == CANTP_TX_WAIT_FC) {
        ch->txBs = bs;
        ch->txStmin = stmin;
        ch->txState = CANTP_TX_WAIT_CF; // CHUYỂN SANG ĐÈN XANH RẢI
    }
}
```

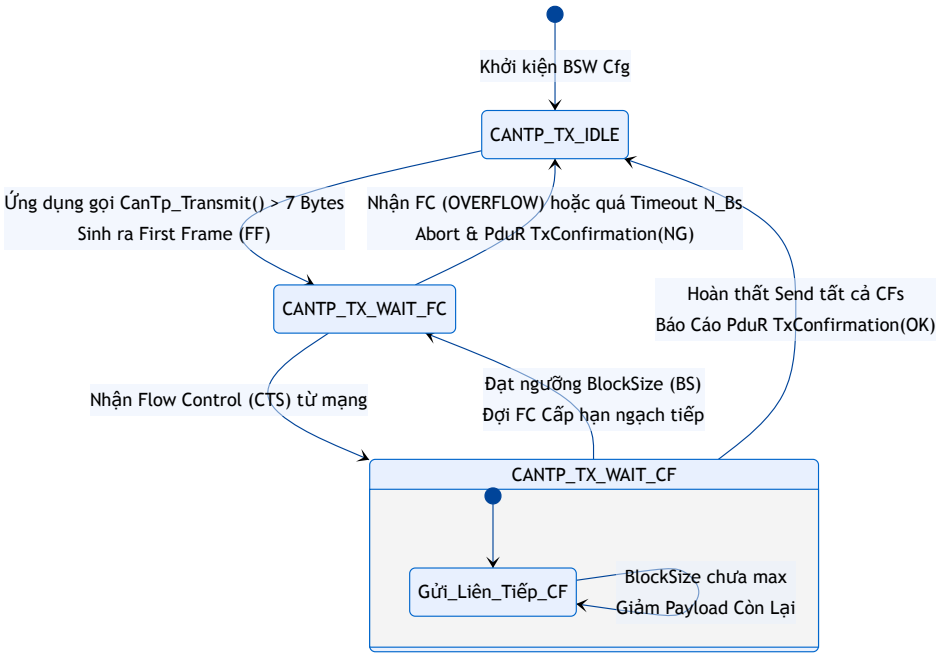
5.4 Lưới Đạn Consecutive Frames!

Nhờ vòng lặp vô hạn gọi Task `CanTp_MainFunction()`, mỗi khi State nằm ở `CANTP_TX_WAIT_CF`, một frame 8 Bytes được sinh ra, đắp nilon dán tem SN (chạy vòng quanh 0-F), rồi chèn tối đa 7 bytes dữ liệu liên tiếp. Bức xạ vòng tròn liên tục cho đến khi `txBytesRemaining` = 0. Hoàn toàn tự thân vận động không cần App.

---

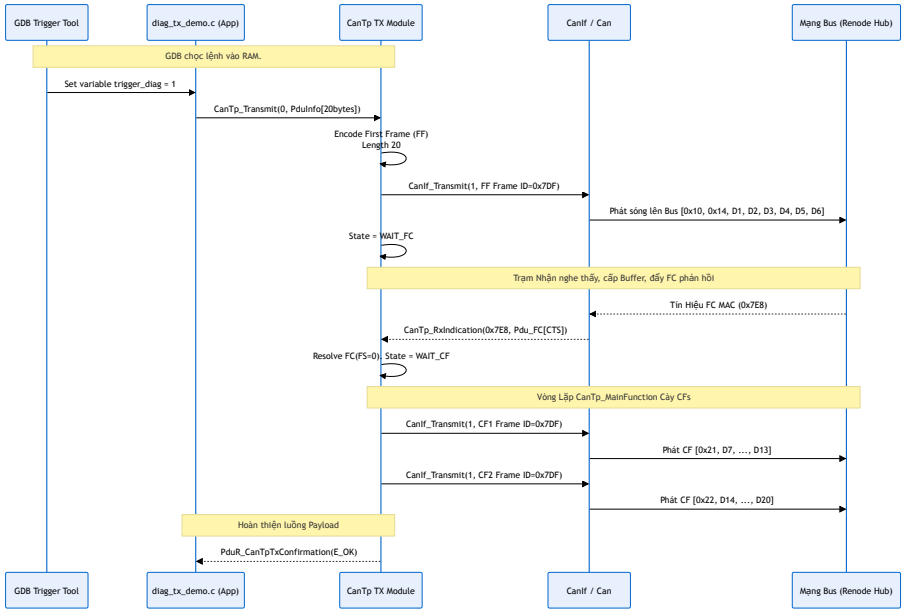
6. Biểu đồ kỹ thuật: State Machine & Sequence

Hình 1. CanTp State Machine - Node Truyền (TX Flow)



Hệ máy trạng thái quản lý chuỗi đời truyền tín hiệu chẩn đoán bằng Phân Mảnh (Segmentation).

Hình 2. Sequence Diagram - Phân Mảnh Truyền UDS từ Node GDB



GDB kích hoạt quá trình, TX nện First Frame, mạng dội Flow Control và sau đó là đợt oanh tạc Consecutive Frames!